Python Programming Fundamentals

Validation and Defensive Programming

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1.1 Garbage In, Garbage Out

Without validation, bad data silently corrupts our program:

```
# No validation — these "work" but make no sense
p1 = Product("Apple", -5.00, 101)   # negative price!
p2 = Product("",       1.99, 102)   # empty name!
p3 = Product("Milk",   0.00, 103)   # free milk?
```

Defensive programming means checking inputs **before** storing them, so problems are caught early with clear error messages.

Outline

2.1 Price must be greater than zero

```
class Product:
    def set_price(self, price: float):
        if price <= 0:
            raise ValueError(
                f"Price must be greater than 0, got {price}"
            )
        self.__price = price

    def __init__(self, name: str, price: float, product_id: int):
        self.__name = ""
        self.__price = 0.0
        self.__product_id = product_id
        self.set_name(name)    # validate via setter
        self.set_price(price)  # validate via setter
```

Outline

3.1 Name must not be empty

```
class Product:
    def set_name(self, name: str):
        name = name.strip()           # remove whitespace
        if not name:
            raise ValueError(
                "Product name cannot be empty"
            )
        self.__name = name
```

`.strip()` removes leading/trailing spaces, so `"   "` becomes `""` which fails the check — good!

Outline

4.1 Python's built-in exception for bad values

```
raise ValueError("message describing the problem")
```

- `ValueError` is the standard Python exception for invalid values
- `raise` immediately stops execution of the current function
- The caller must handle it or the program crashes with a helpful message

Other useful exceptions:

- `TypeError` — wrong type passed in
- `IndexError` — index out of range
- `KeyError` — key not found in dict

Outline

5. Handling Exceptions with try/except

```python
from product import Product

def main():
    try:
        p = Product("Apple", -1.99, 101)  # will raise!
        print(p)
    except ValueError as e:
        print(f"Could not create product: {e}")

    # Program continues here
    print("Program still running...")

if __name__ == "__main__":
    main()
```

Output:

5. Handling Exceptions with try/except

```
Could not create product: Price must be greater than 0, got -1.99
Program still running...
```

Outline

6. Defensive Programming in the Driver

```
def create_product_safely(name, price, pid):
    try:
        return Product(name, price, pid)
    except ValueError as e:
        print(f"  [ERROR] {e}")
        return None

def main():
    products_data = [
        ("Apple",  1.99, 101),
        ("",       2.50, 102),   # bad name
        ("Milk",  -1.00, 103),   # bad price
        ("Bread",  2.00, 104),
    ]
    products = []
    for name, price, pid in products_data:
```

6. Defensive Programming in the Driver

```
    p = create_product_safely(name, price, pid)
    if p:
        products.append(p)
print(f"\n{len(products)} valid products created")
```

Outline

7. Updated Product Class with Validation

```
class Product:
    def __init__(self, name: str, price: float,
                 product_id: int):
        self.__product_id = product_id
        self.__name = ""
        self.__price = 0.0
        self.set_name(name)
        self.set_price(price)

    def set_name(self, name: str):
        name = name.strip()
        if not name:
            raise ValueError("Name cannot be empty")
        self.__name = name

    def set_price(self, price: float):
```

7. Updated Product Class with Validation

```
if price <= 0:
    raise ValueError(
        f"Price must be > 0, got {price}")
self.__price = price
```

Outline

8. Summary

Concept	Purpose
`raise ValueError`	Signal that a value is invalid
`try/except`	Handle errors gracefully
Setter validation	Enforce rules when data is set
`name.strip()`	Clean whitespace before checking
Call setters in `__init__`	Validate on construction too

Thanks for Watching - Any questions?